MANUAL TÉCNICO DEL SISTEMA — SAE QR

Arquitectura, Stack Tecnológico, Base de Datos, APIs, Diagramas y Despliegue (Laravel 12)

1. Arquitectura General del Sistema

El sistema SAE QR está construido bajo el patrón Modelo-Vista-Controlador (MVC) en Laravel 12, complementado con endpoints asíncronos JSON (Fetch / AJAX) para la emisión y validación de códigos QR dinámicos, confirmación de lectura en tiempo real y búsquedas sobre padrones estudiantiles.

- Modelo Universitario Modular: La entidad SesionClase vincula de forma ternaria profesor_id + grupo_id + materia_id. La tabla matriculas aplica la restricción única compuesta UNIQUE(user_id, grupo_id, materia_id).
- Control de Acceso RBAC: El middleware CheckRol intercepta cada petición autenticada y valida el rol (admin, profesor, alumno).
- Seguridad Criptográfica del QR (HMAC-SHA256): Los tokens QR tienen el formato compacto {user_id}.{timestamp_exp}.{firma_16_hex} firmados con APP_KEY y validados con hash_equals() y vigencia estricta de 60 segundos.

2. Stack Tecnológico Completo

- Backend: PHP ^8.2 (8.2.12), Laravel Framework ^12.0, Eloquent ORM, Carbon, Laravel Sanctum ^4.0.
- Base de Datos: MySQL / MariaDB (Motor InnoDB, codificación utf8mb4_unicode_ci, puerto 3307 en XAMPP).
- Frontend y Librerías JS: Blade Templates, Bootstrap 5.3.0, Bootstrap Icons 1.10.5, QRCode.js 1.0.0 (generador QR cliente), html5-qrcode + BarCodeDetector API (escáner de cámara), Chart.js 4.4.1 (gráficas analíticas).

3. Diccionario de Datos Relacional (8 Tablas de Dominio)

- 1. users: id (PK), name, email (UNIQUE), password (Bcrypt), rol ENUM(admin, profesor, alumno), matricula_codigo (UNIQUE, nullable).
- 2. codigos_docentes: id (PK), codigo (UNIQUE), descripcion_nota, usado (BOOLEAN), usado_por_id (FK users), creado_por_id (FK users), usado_en.
- 3. grupos: id (PK), nombre_grupo, carrera (VARCHAR 120), grado (Semestre), turno, periodo_lectivo, profesor_id (nullable). UNIQUE(nombre_grupo, turno).
- 4. materias: id (PK), nombre_materia, codigo_materia, grupo_id (FK grupos ON DELETE CASCADE), profesor_id (FK users ON DELETE SET NULL).
- 5. matriculas: id (PK), user_id (FK users), grupo_id (FK grupos), materia_id (FK materias), estado ENUM(pendiente, aceptado, rechazado). UNIQUE(user_id, grupo_id, materia_id).
- 6. sesion_clases: id (PK), profesor_id (FK users), grupo_id (FK grupos), materia_id (FK materias), fecha, hora_inicio, hora_fin, tolerancia_minutos, estado ENUM(activa, finalizada).
- 7. registro_asistencias: id (PK), sesion_clase_id (FK sesion_clases), user_id (FK users), timestamp_escaneo, estado_incidencia ENUM(asistencia, retardo, falta, permiso), observaciones.
- 8. justificantes: id (PK), registro_id (FK registro_asistencias), motivo, comentario_profesor (VARCHAR 500, nullable), documento_url (nullable),

estado_aprobacion ENUM(pendiente, aceptado, rechazado).

4. APIs Internas y Endpoints AJAX Principales

- GET /alumno/qr/generar (QrController@generarTokenQR): Retorna JSON con el token firmado HMAC-SHA256 válido por 60 segundos.
- POST /profesor/qr/validar (QrController@validarEscaneo): Valida firma HMAC, vigencia, sesión activa y matrícula aceptada; inserta asistencia o retardo y devuelve JSON 200 OK.
- GET /alumno/qr/estado (QrController@verificarEscaneoReciente): Endpoint de polling ligero (cada 4 s) que confirma en la pantalla del alumno cuando su QR ha sido escaneado hoy.
- GET /profesor/alumnos/buscar (MatriculaController@buscarAlumnos): Búsqueda asíncrona de alumnos por nombre, matrícula o correo (máx. 15 resultados).
- GET /alumno/grupos/{grupo}/materias (MatriculaController@materiasPorGrupo): Carga dinámica de asignaturas e indicador de inscripción previa por grupo.

5. Flujo Técnico del Pase de Lista QR

```
[Alumno Dashboard] --(1. GET /alumno/qr/generar cada 60s)--> [QrController]
[Alumno Dashboard] <--(2. Token HMAC: id.exp.firma16)------- [QrController]
[Profesor Camara]  --(3. Escanea QR con html5-qrcode)------> [POST /profesor/qr/validar]
[QrController]     --(4. Valida HMAC + Vigencia + Matricula)-> [INSERT registro_asistencias]
[Alumno Dashboard] --(5. Polling GET /alumno/qr/estado)----> [Muestra Confirmacion en Vivo]
```

6. Procedimiento de Instalación y Despliegue (Entorno de Desarrollo)

- Paso 1: Instalar dependencias: Ejecutar "composer install" dentro de c:\xampp\htdocs\sae_qr.
- Paso 2: Configurar archivo .env: Copiar .env.example a .env, ejecutar "php artisan key:generate" y configurar DB_PORT=3307 (o 3306) y DB_DATABASE=sae_qr.
- Paso 3: Ejecutar migraciones y enlace de archivos: Ejecutar "php artisan migrate" y "php artisan storage:link".
- Paso 4: Credenciales de Administrador Inicial: Correo: admin@saeqr.com | Contraseña: admin123.